
14.8 Exercises

Reinforcement

- R-14.1 Draw a simple undirected graph G that has 12 vertices, 18 edges, and 3 connected components.
- R-14.2 If G is a simple undirected graph with 12 vertices and 3 connected components, what is the largest number of edges it might have?
- R-14.3 Draw an adjacency matrix representation of the undirected graph shown in Figure 14.1.
- R-14.4 Draw an adjacency list representation of the undirected graph shown in Figure 14.1.
- R-14.5 Draw a simple, connected, directed graph with 8 vertices and 16 edges such that the in-degree and out-degree of each vertex is 2. Show that there is a single (nonsimple) cycle that includes all the edges of your graph, that is, you can trace all the edges in their respective directions without ever lifting your pencil. (Such a cycle is called an *Euler tour*.)
- R-14.6 Suppose we represent a graph G having n vertices and m edges with the edge list structure. Why, in this case, does the `insertVertex` method run in $O(1)$ time while the `removeVertex` method runs in $O(m)$ time?
- R-14.7 Give pseudocode for performing the operation `insertEdge( $u, v, x$ )` in $O(1)$ time using the adjacency matrix representation.
- R-14.8 Repeat Exercise R-14.7 for the adjacency list representation, as described in the chapter.
- R-14.9 Can edge list E be omitted from the adjacency matrix representation while still achieving the time bounds given in Table 14.1? Why or why not?
- R-14.10 Can edge list E be omitted from the adjacency list representation while still achieving the time bounds given in Table 14.3? Why or why not?
- R-14.11 Would you use the adjacency matrix structure or the adjacency list structure in each of the following cases? Justify your choice.
 - a. The graph has 10,000 vertices and 20,000 edges, and it is important to use as little space as possible.
 - b. The graph has 10,000 vertices and 20,000,000 edges, and it is important to use as little space as possible.
 - c. You need to answer the query `getEdge( $u, v$ )` as fast as possible, no matter how much space you use.
- R-14.12 In order to verify that all of its nontree edges are back edges, redraw the graph from Figure 14.8b so that the DFS tree edges are drawn with solid lines and oriented downward, as in a standard portrayal of a tree, and with all nontree edges drawn using dashed lines.

- R-14.13 Explain why the DFS traversal runs in $O(n^2)$ time on an n -vertex simple graph that is represented with the adjacency matrix structure.
- R-14.14 A simple undirected graph is **complete** if it contains an edge between every pair of distinct vertices. What does a depth-first search tree of a complete graph look like?
- R-14.15 Recalling the definition of a complete graph from Exercise R-14.14, what does a breadth-first search tree of a complete graph look like?
- R-14.16 Let G be an undirected graph whose vertices are the integers 1 through 8, and let the adjacent vertices of each vertex be given by the table below:

vertex	adjacent vertices
1	(2, 3, 4)
2	(1, 3, 4)
3	(1, 2, 4)
4	(1, 2, 3, 6)
5	(6, 7, 8)
6	(4, 5, 7)
7	(5, 6, 8)
8	(5, 7)

Assume that, in a traversal of G , the adjacent vertices of a given vertex are returned in the same order as they are listed in the table above.

- Draw G .
 - Give the sequence of vertices of G visited using a DFS traversal starting at vertex 1.
 - Give the sequence of vertices visited using a BFS traversal starting at vertex 1.
- R-14.17 Bob loves foreign languages and wants to plan his course schedule for the following years. He is interested in the following nine language courses: LA15, LA16, LA22, LA31, LA32, LA126, LA127, LA141, and LA169. The course prerequisites are:
- LA15: (none)
 - LA16: LA15
 - LA22: (none)
 - LA31: LA15
 - LA32: LA16, LA31
 - LA126: LA22, LA32
 - LA127: LA16
 - LA141: LA22, LA16
 - LA169: LA32

In what order can Bob take these courses, respecting the prerequisites?

- R-14.18 Compute a topological ordering for the directed graph drawn with solid edges in Figure 14.3d.
- R-14.19 Draw the transitive closure of the directed graph shown in Figure 14.2.

- R-14.20 If the vertices of the graph from Figure 14.11 are ordered as (JFK, LAZ, MIA, BOS, ORD, SFO, DFW), in what order would edges be added to the transitive closure during the Floyd-Warshall algorithm?
- R-14.21 How many edges are in the transitive closure of a graph that consists of a simple directed path of n vertices?
- R-14.22 Given an n -node complete binary tree T , rooted at a given position, consider a directed graph $\vec{G}$ having the nodes of T as its vertices. For each parent-child pair in T , create a directed edge in $\vec{G}$ from the parent to the child. Show that the transitive closure of $\vec{G}$ has $O(n \log n)$ edges.
- R-14.23 Draw a simple, connected, weighted graph with 8 vertices and 16 edges, each with unique edge weights. Identify one vertex as a “start” vertex and illustrate a running of Dijkstra’s algorithm on this graph.
- R-14.24 Show how to modify the pseudocode for Dijkstra’s algorithm for the case when the graph is directed and we want to compute shortest directed paths from the source vertex to all the other vertices.
- R-14.25 Draw a simple, connected, undirected, weighted graph with 8 vertices and 16 edges, each with unique edge weights. Illustrate the execution of the Prim-Jarník algorithm for computing the minimum spanning tree of this graph.
- R-14.26 Repeat the previous problem for Kruskal’s algorithm.
- R-14.27 There are eight small islands in a lake, and the state wants to build seven bridges to connect them so that each island can be reached from any other one via one or more bridges. The cost of constructing a bridge is proportional to its length. The distances between pairs of islands are given in the following table.
- | | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|-----|-----|-----|-----|-----|-----|-----|
| 1 | - | 240 | 210 | 340 | 280 | 200 | 345 | 120 |
| 2 | - | - | 265 | 175 | 215 | 180 | 185 | 155 |
| 3 | - | - | - | 260 | 115 | 350 | 435 | 195 |
| 4 | - | - | - | - | 160 | 330 | 295 | 230 |
| 5 | - | - | - | - | - | 360 | 400 | 170 |
| 6 | - | - | - | - | - | - | 175 | 205 |
| 7 | - | - | - | - | - | - | - | 305 |
| 8 | - | - | - | - | - | - | - | - |

Find which bridges to build to minimize the total construction cost.

- R-14.28 Describe the meaning of the graphical conventions used in Figure 14.9 illustrating a DFS traversal. What do the line thicknesses signify? What do the arrows signify? How about dashed lines?
- R-14.29 Repeat Exercise R-14.28 for Figure 14.8 that illustrates a directed DFS traversal.
- R-14.30 Repeat Exercise R-14.28 for Figure 14.10 that illustrates a BFS traversal.
- R-14.31 Repeat Exercise R-14.28 for Figure 14.11 illustrating the Floyd-Warshall algorithm.

- R-14.32 Repeat Exercise R-14.28 for Figure 14.13 that illustrates the topological sorting algorithm.
- R-14.33 Repeat Exercise R-14.28 for Figures 14.15 and 14.16 illustrating Dijkstra's algorithm.
- R-14.34 Repeat Exercise R-14.28 for Figures 14.20 and 14.21 that illustrate the Prim-Jarník algorithm.
- R-14.35 Repeat Exercise R-14.28 for Figures 14.22 through 14.24 that illustrate Kruskal's algorithm.
- R-14.36 George claims he has a fast way to do path compression in a partition structure, starting at a position p . He puts p into a list L , and starts following parent pointers. Each time he encounters a new position, q , he adds q to L and updates the parent pointer of each node in L to point to q 's parent. Show that George's algorithm runs in $\Omega(h^2)$ time on a path of length h .

Creativity

- C-14.37 Give a Java implementation of the `removeEdge( $e$ )` method for our adjacency map implementation of Section 14.2.5, making sure your implementation works for both directed and undirected graphs. Your method should run in $O(1)$ time.
- C-14.38 Suppose we wish to represent an n -vertex graph G using the edge list structure, assuming that we identify the vertices with the integers in the set $\{0, 1, \dots, n-1\}$. Describe how to implement the collection E to support $O(\log n)$ -time performance for the `getEdge( $u, v$ )` method. How are you implementing the method in this case?
- C-14.39 Let T be the spanning tree rooted at the start vertex produced by the depth-first search of a connected, undirected graph G . Argue why every edge of G not in T goes from a vertex in T to one of its ancestors, that is, it is a **back edge**.
- C-14.40 Our solution to reporting a path from u to v in Code Fragment 14.6 could be made more efficient in practice if the DFS process ended as soon as v is discovered. Describe how to modify our code base to implement this optimization.
- C-14.41 Let G be an undirected graph with n vertices and m edges. Describe an $O(n+m)$ -time algorithm for traversing each edge of G exactly once in each direction.
- C-14.42 Implement an algorithm that returns a cycle in a directed graph $\vec{G}$, if one exists.
- C-14.43 Write a method, `components( $G$ )`, for undirected graph G , that returns a dictionary mapping each vertex to an integer that serves as an identifier for its connected component. That is, two vertices should be mapped to the same identifier if and only if they are in the same connected component.
- C-14.44 Say that a maze is **constructed correctly** if there is one path from the start to the finish, the entire maze is reachable from the start, and there are no loops around any portions of the maze. Given a maze drawn in an $n \times n$ grid, how can we determine if it is constructed correctly? What is the running time of this algorithm?

- C-14.45 Computer networks should avoid single points of failure, that is, network vertices that can disconnect the network if they fail. We say an undirected, connected graph G is **biconnected** if it contains no vertex whose removal would divide G into two or more connected components. Give an algorithm for adding at most n edges to a connected graph G , with $n \geq 3$ vertices and $m \geq n - 1$ edges, to guarantee that G is biconnected. Your algorithm should run in $O(n + m)$ time.
- C-14.46 Explain why all nontree edges are cross edges, with respect to a BFS tree constructed for an undirected graph.
- C-14.47 Explain why there are no forward nontree edges with respect to a BFS tree constructed for a directed graph.
- C-14.48 Show that if T is a BFS tree produced for a connected graph G , then, for each vertex v at level i , the path of T between s and v has i edges, and any other path of G between s and v has at least i edges.
- C-14.49 Justify Proposition 14.16.
- C-14.50 Provide an implementation of the BFS algorithm that uses a FIFO queue, rather than a level-by-level formulation, to manage vertices that have been discovered until the time when their neighbors are considered.
- C-14.51 A graph G is **bipartite** if its vertices can be partitioned into two sets X and Y such that every edge in G has one end vertex in X and the other in Y . Design and analyze an efficient algorithm for determining if an undirected graph G is bipartite (without knowing the sets X and Y in advance).
- C-14.52 An **Euler tour** of a directed graph $\vec{G}$ with n vertices and m edges is a cycle that traverses each edge of $\vec{G}$ exactly once according to its direction. Such a tour always exists if $\vec{G}$ is connected and the in-degree equals the out-degree of each vertex in $\vec{G}$. Describe an $O(n + m)$ -time algorithm for finding an Euler tour of such a directed graph $\vec{G}$.
- C-14.53 A company named RT&T has a network of n switching stations connected by m high-speed communication links. Each customer's phone is directly connected to one station in his or her area. The engineers of RT&T have developed a prototype video-phone system that allows two customers to see each other during a phone call. In order to have acceptable image quality, however, the number of links used to transmit video signals between the two parties cannot exceed 4. Suppose that RT&T's network is represented by a graph. Design an efficient algorithm that computes, for each station, the set of stations it can reach using no more than 4 links.
- C-14.54 The time delay of a long-distance call can be determined by multiplying a small fixed constant by the number of communication links on the telephone network between the caller and callee. Suppose the telephone network of a company named RT&T is a tree. The engineers of RT&T want to compute the maximum possible time delay that may be experienced in a long-distance call. Given a tree T , the *diameter* of T is the length of a longest path between two nodes of T . Give an efficient algorithm for computing the diameter of T .

- C-14.55 Tamarindo University and many other schools worldwide are doing a joint project on multimedia. A computer network is built to connect these schools using communication links that form a tree. The schools decide to install a file server at one of the schools to share data among all the schools. Since the transmission time on a link is dominated by the link setup and synchronization, the cost of a data transfer is proportional to the number of links used. Hence, it is desirable to choose a “central” location for the file server. Given a tree T and a node v of T , the *eccentricity* of v is the length of a longest path from v to any other node of T . A node of T with minimum eccentricity is called a *center* of T .
- Design an efficient algorithm that, given an n -node tree T , computes a center of T .
 - Is the center unique? If not, how many distinct centers can a tree have?
- C-14.56 Say that an n -vertex directed acyclic graph $\vec{G}$ is **compact** if there is some way of numbering the vertices of $\vec{G}$ with the integers from 0 to $n - 1$ such that $\vec{G}$ contains the edge (i, j) if and only if $i < j$, for all i, j in $[0, n - 1]$. Give an $O(n^2)$ -time algorithm for detecting if $\vec{G}$ is compact.
- C-14.57 Let $\vec{G}$ be a weighted directed graph with n vertices. Design a variation of Floyd-Warshall’s algorithm for computing the lengths of the shortest paths from each vertex to every other vertex in $O(n^3)$ time.
- C-14.58 Design an efficient algorithm for finding a **longest** directed path from a vertex s to a vertex t of an acyclic weighted directed graph $\vec{G}$. Specify the graph representation used and any auxiliary data structures used. Also, analyze the time complexity of your algorithm.
- C-14.59 An independent set of an undirected graph $G = (V, E)$ is a subset I of V such that no two vertices in I are adjacent. That is, if u and v are in I , then (u, v) is not in E . A **maximal independent set** M is an independent set such that, if we were to add any additional vertex to M , then it would not be independent any more. Every graph has a maximal independent set. (Can you see this? This question is not part of the exercise, but it is worth thinking about.) Give an efficient algorithm that computes a maximal independent set for a graph G . What is this method’s running time?
- C-14.60 Give an example of an n -vertex simple graph G that causes Dijkstra’s algorithm to run in $\Omega(n^2 \log n)$ time when its implemented with a heap.
- C-14.61 Give an example of a weighted directed graph $\vec{G}$ with negative-weight edges, but no negative-weight cycle, such that Dijkstra’s algorithm incorrectly computes the shortest-path distances from some start vertex s .
- C-14.62 Our implementation of `shortestPathLengths` in Code Fragment 14.13 relies on use of “infinity” as a numeric value, to represent the distance bound for vertices that are not (yet) known to be reachable from the source. Reimplement that method without such a sentinel, so that vertices, other than the source, are not added to the priority queue until it is evident that they are reachable.

C-14.63 Consider the following greedy strategy for finding a shortest path from vertex *start* to vertex *goal* in a given connected graph.

- 1: Initialize *path* to *start*.
- 2: Initialize set *visited* to $\{start\}$.
- 3: If *start*=*goal*, return *path* and exit. Otherwise, continue.
- 4: Find the edge (*start*,*v*) of minimum weight such that *v* is adjacent to *start* and *v* is not in *visited*.
- 5: Add *v* to *path*.
- 6: Add *v* to *visited*.
- 7: Set *start* equal to *v* and go to step 3.

Does this greedy strategy always find a shortest path from *start* to *goal*? Either explain intuitively why it works, or give a counterexample.

C-14.64 Show that if all the weights in a connected weighted graph *G* are distinct, then there is exactly one minimum spanning tree for *G*.

C-14.65 An old MST method, called *Barůvka's algorithm*, works as follows on a graph *G* having *n* vertices and *m* edges with distinct weights:

```

Let T be a subgraph of G initially containing just the vertices in V.
while T has fewer than  $n - 1$  edges do
  for each connected component  $C_i$  of T do
    Find the lowest-weight edge (u,v) in E with u in  $C_i$  and v not in  $C_i$ .
    Add (u,v) to T (unless it is already in T).
return T

```

Prove that this algorithm is correct and that it runs in $O(m \log n)$ time.

C-14.66 Let *G* be a graph with *n* vertices and *m* edges such that all the edge weights in *G* are integers in the range $[1, n]$. Give an algorithm for finding a minimum spanning tree for *G* in $O(m \log^* n)$ time.

C-14.67 Consider a diagram of a telephone network, which is a graph *G* whose vertices represent switching centers, and whose edges represent communication lines joining pairs of centers. Edges are marked by their bandwidth, and the bandwidth of a path is equal to the lowest bandwidth among the path's edges. Give an algorithm that, given a network and two switching centers *a* and *b*, outputs the maximum bandwidth of a path between *a* and *b*.

C-14.68 NASA wants to link *n* stations spread over the country using communication channels. Each pair of stations has a different bandwidth available, which is known a priori. NASA wants to select $n - 1$ channels (the minimum possible) in such a way that all the stations are linked by the channels and the total bandwidth (defined as the sum of the individual bandwidths of the channels) is maximum. Give an efficient algorithm for this problem and determine its worst-case time complexity. Consider the weighted graph $G = (V, E)$, where *V* is the set of stations and *E* is the set of channels between the stations. Define the weight $w(e)$ of an edge *e* in *E* as the bandwidth of the corresponding channel.

C-14.69 Inside the Castle of Asymptopia there is a maze, and along each corridor of the maze there is a bag of gold coins. The amount of gold in each bag varies. A noble knight, named Sir Paul, will be given the opportunity to walk through the maze, picking up bags of gold. He may enter the maze only through a door marked “ENTER” and exit through another door marked “EXIT.” While in the maze he may not retrace his steps. Each corridor of the maze has an arrow painted on the wall. Sir Paul may only go down the corridor in the direction of the arrow. There is no way to traverse a “loop” in the maze. Given a map of the maze, including the amount of gold in each corridor, describe an algorithm to help Sir Paul pick up the most gold.

C-14.70 Suppose you are given a *timetable*, which consists of:

- A set $\mathcal{A}$ of n airports, and for each airport a in $\mathcal{A}$, a minimum connecting time $c(a)$.
- A set $\mathcal{F}$ of m flights, and the following, for each flight f in $\mathcal{F}$:
 - Origin airport $a_1(f)$ in $\mathcal{A}$
 - Destination airport $a_2(f)$ in $\mathcal{A}$
 - Departure time $t_1(f)$
 - Arrival time $t_2(f)$

Describe an efficient algorithm for the flight scheduling problem. In this problem, we are given airports a and b , and a time t , and we wish to compute a sequence of flights that allows one to arrive at the earliest possible time in b when departing from a at or after time t . Minimum connecting times at intermediate airports must be observed. What is the running time of your algorithm as a function of n and m ?

C-14.71 Suppose we are given a directed graph $\vec{G}$ with n vertices, and let M be the $n \times n$ adjacency matrix corresponding to $\vec{G}$.

- a. Let the product of M with itself (M^2) be defined, for $1 \leq i, j \leq n$, as follows:

$$M^2(i, j) = M(i, 1) \odot M(1, j) \oplus \cdots \oplus M(i, n) \odot M(n, j),$$

where “ $\oplus$ ” is the boolean **or** operator and “ $\odot$ ” is boolean **and**. Given this definition, what does $M^2(i, j) = 1$ imply about the vertices i and j ? What if $M^2(i, j) = 0$?

- b. Suppose M^4 is the product of M^2 with itself. What do the entries of M^4 signify? How about the entries of $M^5 = (M^4)(M)$? In general, what information is contained in the matrix M^p ?
- c. Now suppose that $\vec{G}$ is weighted and assume the following:
- 1: for $1 \leq i \leq n$, $M(i, i) = 0$.
 - 2: for $1 \leq i, j \leq n$, $M(i, j) = \text{weight}(i, j)$ if (i, j) is in E .
 - 3: for $1 \leq i, j \leq n$, $M(i, j) = \infty$ if (i, j) is not in E .

Also, let M^2 be defined, for $1 \leq i, j \leq n$, as follows:

$$M^2(i, j) = \min\{M(i, 1) + M(1, j), \dots, M(i, n) + M(n, j)\}.$$

If $M^2(i, j) = k$, what may we conclude about the relationship between vertices i and j ?

- C-14.72 Karen has a new way to do path compression in a tree-based union/find partition data structure starting at a position p . She puts all the positions that are on the path from p to the root in a set S . Then she scans through S and sets the parent pointer of each position in S to its parent's parent pointer (recall that the parent pointer of the root points to itself). If this pass changed the value of any position's parent pointer, then she repeats this process, and goes on repeating this process until she makes a scan through S that does not change any position's parent value. Show that Karen's algorithm is correct and analyze its running time for a path of length h .

Projects

- P-14.73 Use an adjacency matrix to implement a class supporting a simplified graph ADT that does not include update methods. Your class should include a constructor method that takes two collections—a collection V of vertex elements and a collection E of pairs of vertex elements—and produces the graph G that these two collections represent.
- P-14.74 Implement the simplified graph ADT described in Exercise P-14.73, using the edge list structure.
- P-14.75 Implement the simplified graph ADT described in Exercise P-14.73, using the adjacency list structure.
- P-14.76 Extend the class of Exercise P-14.75 to support the update methods of the graph ADT.
- P-14.77 Design an experimental comparison of repeated DFS traversals versus the Floyd-Warshall algorithm for computing the transitive closure of a directed graph.
- P-14.78 Develop a Java implementation of the Prim-Jarník algorithm for computing the minimum spanning tree of a graph.
- P-14.79 Perform an experimental comparison of two of the minimum spanning tree algorithms discussed in this chapter (Kruskal and Prim-Jarník). Develop an extensive set of experiments to test the running times of these algorithms using randomly generated graphs.
- P-14.80 One way to construct a *maze* starts with an $n \times n$ grid such that each grid cell is bounded by four unit-length walls. We then remove two boundary unit-length walls, to represent the start and finish. For each remaining unit-length wall not on the boundary, we assign a random value and create a graph G , called the *dual*, such that each grid cell is a vertex in G and there is an edge joining the vertices for two cells if and only if the cells share a common wall. The weight of each edge is the weight of the corresponding wall. We construct the maze by finding a minimum spanning tree T for G and removing all the walls corresponding to edges in T . Write a program that uses this algorithm to generate mazes and then solves them. Minimally, your program should draw the maze and, ideally, it should visualize the solution as well.

- P-14.81 Write a program that builds the routing tables for the nodes in a computer network, based on shortest-path routing, where path distance is measured by hop count, that is, the number of edges in a path. The input for this problem is the connectivity information for all the nodes in the network, as in the following example:

241.12.31.14: 241.12.31.15 241.12.31.18 241.12.31.19

which indicates three network nodes that are connected to 241.12.31.14, that is, three nodes that are one hop away. The routing table for the node at address A is a set of pairs (B, C) , which indicates that, to route a message from A to B , the next node to send to (on the shortest path from A to B) is C . Your program should output the routing table for each node in the network, given an input list of node connectivity lists, each of which is input in the syntax as shown above, one per line.

Chapter Notes

The depth-first search method is a part of the “folklore” of computer science, but Hopcroft and Tarjan [46, 87] are the ones who showed how useful this algorithm is for solving several different graph problems. Knuth [60] discusses the topological sorting problem. The simple linear-time algorithm that we describe for determining if a directed graph is strongly connected is due to Kosaraju. The Floyd-Warshall algorithm appears in a paper by Floyd [34] and is based upon a theorem of Warshall [94].

The first known minimum spanning tree algorithm is due to Barůvka [9], and was published in 1926. The Prim-Jarník algorithm was first published in Czech by Jarník [51] in 1930 and in English in 1957 by Prim [79]. Kruskal published his minimum spanning tree algorithm in 1956 [63]. The reader interested in further study of the history of the minimum spanning tree problem is referred to the paper by Graham and Hell [41]. The current asymptotically fastest minimum spanning tree algorithm is a randomized method of Karger, Klein, and Tarjan [53] that runs in $O(m)$ expected time. Dijkstra [30] published his single-source, shortest-path algorithm in 1959. The running time for the Prim-Jarník algorithm, and also that of Dijkstra’s algorithm, can actually be improved to be $O(n \log n + m)$ by implementing the queue Q with either of two more sophisticated data structures, the “Fibonacci Heap” [36] or the “Relaxed Heap” [32].

To learn about different algorithms for drawing graphs, please see the book chapter by Tamassia and Liotta [85] and the book by Di Battista, Eades, Tamassia and Tollis [29]. The reader interested in further study of graph algorithms is referred to the books by Ahuja, Magnanti, and Orlin [7], Cormen, Leiserson, Rivest and Stein [25], Mehlhorn [72], and Tarjan [88], and the book chapter by van Leeuwen [90].